

Puzzle Problem

Name: Hemalakshmi H (192571049)

Aim:

To develop a Python program that solves the 8-Puzzle (Sliding Puzzle) using Breadth-First Search (BFS) and displays the sequence of steps required to reach the goal state.

Algorithm:

Step 1: Start with the initial puzzle state (3×3 matrix with one blank represented as 0).

Step 2: Check if the current state is the goal state. Step 3: Find the position of the blank tile (0). Step 4: Generate all valid neighboring states by moving the blank tile:

- Up
 - Down
 - Left
 - Right
- Step 5: Add all unvisited neighboring states to the BFS queue. Step 6: Mark the current state as visited. Step 7: Repeat Steps 2–6 until the goal state is found. Step 8: Return the complete path from initial state to goal state.

Source Code:

```
from collections import deque
```

```
# Initial and goal states
```

```
initial = (  
    (1, 2, 3),  
    (4, 0, 6),  
    (7, 5, 8)  
)
```

```
goal = (  
    (1, 2, 3),  
    (4, 5, 6),  
    (7, 8, 0)  
)
```

```

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)
    directions = [(-1,0),(1,0),(0,-1),(0,1)] # up, down, left, right

    for dx, dy in directions:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

def bfs(initial, goal):
    queue = deque()
    queue.append((initial, []))
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state in visited:
            continue
        visited.add(state)

```

```
        path = path + [state]

    if state == goal:
        return path

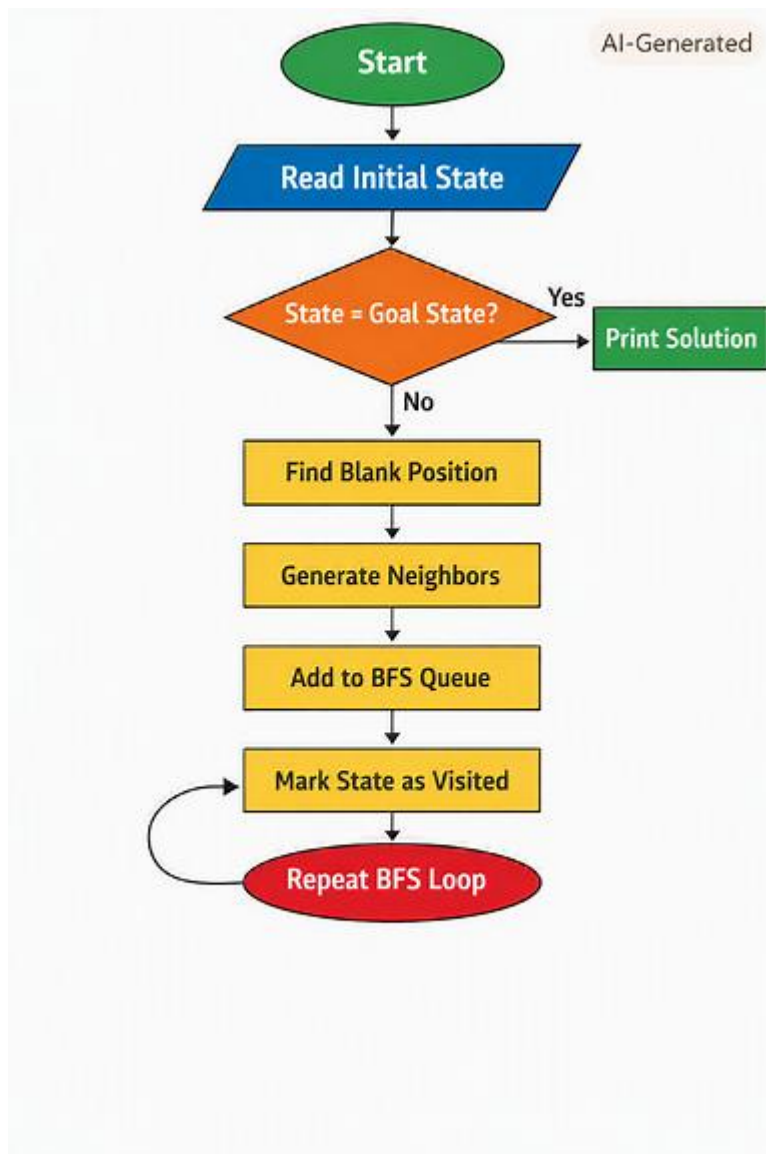
    for neighbor in get_neighbors(state):
        if neighbor not in visited:
            queue.append((neighbor, path))

    return None

solution = bfs(initial, goal)

print("Steps to solve the 8 Puzzle:\n")
step = 1
for state in solution:
    print("Step", step)
    for row in state:
        for value in row:
            print("_" if value == 0 else value, end=" ")
        print()
    print()
    step += 1
```

Flowchart:



Output:

```
===== RESTART: C:\Users\hemal\OneDrive\Documents\SIMATS\AI\puzzle_AI.py =====
Steps to solve the 8 Puzzle:

Step 1
1 2 3
4 _ 6
7 5 8

Step 2
1 2 3
4 5 6
7 _ 8

Step 3
1 2 3
4 5 6
7 8 _
```

Result:

The 8-Puzzle program successfully solved the given initial puzzle configuration using the Breadth-First Search (BFS) algorithm. The program generated all possible valid moves by sliding the blank tile and explored each state level-by-level until it reached the goal state.

During execution, the program displayed every intermediate step clearly. At each step, the blank tile (0) was moved either up, down, left, or right to bring the puzzle closer to the goal arrangement. Finally, the program reached the goal state where all the tiles were arranged in the correct order and the blank space was positioned at the last cell.

Thus, the BFS algorithm correctly found the shortest sequence of moves required to solve the 8-Puzzle problem.